

Cynix: Documentação de Software Educacional

Diedson Kenned Guiana Mota

Oficina de Desenvolvimento de Software Educacional 1

2026

ESCOPO E VISÃO DO PRODUTO

1. Visão geral do produto

O Cynix é uma plataforma web educacional voltada para à aprendizagem inicial de programação em C. O sistema oferece explicações curtas, exemplos simples, quizzes e um laboratório de código no navegador. A ideia principal é permitir que o estudante pratique sem precisar instalar compiladores ou configurar programas no computador.

A finalidade educacional do Cynix é apoiar o ensino de lógica de programação, comandos de entrada e saída, variáveis, operadores, estruturas condicionais e laços de repetição. O estudante aprende lendo, testando e corrigindo seus próprios erros com apoio do sistema.

O sistema foi pensado para estudantes do início de cursos da área de Computação, especialmente aqueles que chegam ao primeiro semestre com pouco ou nenhum contato com programação. Também pode ser usado por professores como apoio nas aulas de Introdução à Programação, Algoritmos ou disciplinas semelhantes.

O uso principal será em laboratório de informática, sala de aula com notebooks ou estudo em casa. Como funciona no navegador, o estudante consegue acessar a plataforma em computadores diferentes, sem depender de instalação local.

O diferencial do Cynix é reduzir a barreira inicial. Em vez de começar pela instalação de ferramentas, o estudante começa pelo raciocínio: entende o problema, escreve uma solução simples, executa testes e recebe uma orientação clara sobre o que precisa corrigir.

2. Delimitação do escopo

O Cynix tem um foco bem definido: apoiar os primeiros passos em programação estruturada com a linguagem C. Por isso, ele não resolve todos os problemas ligados ao ensino de programação e não pretende substituir a atuação do professor.

- O sistema não ensina instalação de compiladores, configuração de IDEs, uso de Git/GitHub ou publicação de sistemas.
- Não é adequado para projetos grandes, desenvolvimento profissional de software ou sistemas com muitas bibliotecas externas.
- Ficam fora da proposta temas como programação orientada a objetos, desenvolvimento mobile, banco de dados avançado e inteligência artificial.
- A primeira versão não terá chat, fórum em tempo real, integração com sistema acadêmico ou depurador visual avançado.
- O limite pedagógico está nos conteúdos básicos de algoritmos: variáveis, entrada e saída, operadores, decisões, repetições e pequenos problemas com vetores.
- A plataforma não será tratada como jogo educacional. Ela pode usar indicadores de progresso, mas a aprendizagem continua sendo o centro da proposta.

3. Problema educacional

A dificuldade concreta que o sistema busca enfrentar é a entrada difícil na programação. Muitos estudantes chegam à disciplina sem base em lógica, matemática e interpretação de problemas. Além disso, parte do tempo inicial costuma ser gasto com instalação de programas, erros de ambiente e dúvidas técnicas que não estão diretamente ligadas ao conteúdo da aula.

Na prática, isso pode fazer o aluno sentir que programar é difícil antes mesmo de compreender os primeiros conceitos. Quando o erro aparece em uma mensagem técnica demais, o estudante tende a copiar soluções prontas, desistir ou depender totalmente do professor.

A literatura sobre ensino de programação aponta que as disciplinas iniciais de algoritmos costumam apresentar dificuldades frequentes, ligadas à compreensão da lógica, da sintaxe e do próprio processo de resolver problemas por meio de código. Trabalhos como Moraes, Mendes Neto e Osório (2020) e Ambrósio et al. (2011) ajudam a justificar a necessidade de ferramentas com feedback mais claro e prática orientada.

Como dado local de uso, a proposta considera uma aplicação inicial com diagnóstico da turma na primeira aula. Esse diagnóstico pode verificar se os estudantes já programaram antes, se sabem o que é algoritmo, se conseguem interpretar um enunciado simples e se possuem acesso regular a computador ou notebook.

As principais consequências do problema são: desmotivação nas primeiras semanas, atrasos no conteúdo, aumento da reprovação, dependência excessiva do professor e dificuldade para avançar para disciplinas posteriores do curso.

4. Público-alvo e contexto de uso

4.1 Caracterização do estudante

Aspecto	Descrição
Idade	Em geral, entre 17 e 25 anos.
Nível escolar	Primeiros períodos de cursos superiores na área de Computação.
Conhecimento prévio	Uso comum de tecnologia no dia a dia, mas pouco contato com programação.
Dificuldades possíveis	Interpretação de enunciados, raciocínio lógico, matemática básica, medo de errar e pouca paciência com mensagens técnicas.

4.3 Infraestrutura

Item	Necessidade
Dispositivos	Computadores de laboratório, notebooks pessoais ou desktops com navegador atualizado.
Conectividade	Internet para carregar a plataforma. Após o carregamento, as atividades principais podem funcionar localmente.
Sistema operacional	Windows, Linux ou macOS, pois o acesso ocorre pelo navegador.
Restrição técnica	O aluno pode não ter permissão para instalar programas no computador, por isso o sistema evita dependência de instalação.

4.4 Acessibilidade

- Uso de HTML semântico para melhorar a leitura por leitores de tela.
- Textos curtos e enunciados divididos em partes: contexto, tarefa e exemplo.
- Modo de alto contraste e tamanho de fonte ajustável.
- Evitar excesso de animações, pop-ups e distrações visuais.
- Feedback por texto, não apenas por cor, para não prejudicar estudantes daltônicos.
- Controle de velocidade em vídeos ou animações, caso sejam adicionados em versões futuras.

5. Conteúdo alinhado à BNCC

Mesmo sendo pensado para o Ensino Superior, o sistema dialoga com a BNCC e com o complemento de Computação porque trabalha pensamento computacional, cultura digital e resolução de problemas.

Item	Alinhamento
Área relacionada	Matemática e suas Tecnologias / Computação.
Componente	Pensamento Computacional e Programação.

Item	Alinhamento
Competência Geral 2	Exercitar a curiosidade intelectual, formular hipóteses, testar soluções e resolver problemas.
Competência Geral 5	Usar tecnologias digitais de forma crítica, significativa e responsável.
Habilidade de Computação	EM13CO01 - compreender, modificar e criar algoritmos usando estruturas de controle e variáveis.

6. Modelo de Design Instrucional

O modelo escolhido é o ADDIE, formado pelas etapas de Análise, Design, Desenvolvimento, Implementação e Avaliação. Ele foi escolhido porque ajuda a organizar a criação da plataforma de maneira simples e gradual.

Etapa do ADDIE	Aplicação no Cynix
Análise	Levantar as dificuldades dos estudantes e o contexto de uso da disciplina.
Design	Planejar a sequência de conteúdo, atividades, feedback e avaliação.
Desenvolvimento	Criar as telas, o editor de código, os quizzes e o validador das respostas.
Implementação	Usar a plataforma em aula ou como atividade extraclasse.
Avaliação	Verificar notas, erros comuns, progresso e opinião dos estudantes.

7. Objetivos de aprendizagem

ID	Objetivo
OA01	Identificar comandos de entrada e saída em C, acertando pelo menos 80% de um quiz inicial dentro da plataforma.
OA02	Declarar variáveis dos tipos int, float e char em pequenos exemplos, sem erro de sintaxe nas atividades práticas.
OA03	Resolver problemas simples com operadores aritméticos em C, passando nos testes automáticos do laboratório virtual.
OA04	Aplicar estruturas condicionais para tomar decisões simples, com pelo menos 80% de acerto na avaliação da unidade.
OA05	Corrigir erros apontados pelo feedback do sistema, enviando uma nova versão funcional do código.

8. Resultados esperados

Resultado	Variável impactada	Indicador	Instrumento	Critério de sucesso	Momento
R01	Aprendizagem inicial de lógica	Percentual de estudantes com nota igual ou maior que 80.	Avaliação final da unidade.	Pelo menos 70% da turma atinge 80 pontos.	Final da primeira unidade.
R02	Autonomia para praticar	Quantidade de tentativas resolvidas sem ajuda direta do professor.	Registro local de tentativas.	A maioria conclui pelo menos 2 desafios práticos.	Durante o laboratório.
R03	Compreensão dos erros	Redução de erros repetidos de sintaxe.	Histórico de feedback do sistema.	Diminuição dos mesmos erros após as dicas.	Após cada atividade.
R04	Engajamento com a disciplina	Acesso e conclusão das unidades propostas.	Tela de progresso.	Pelo menos 75% conclui a unidade inicial.	Fim do ciclo de uso.

9. Escopo técnico

O Cynix será uma aplicação web simples. A primeira versão pode ser feita com HTML5, CSS3 e JavaScript. O editor de código pode usar uma biblioteca de edição, como CodeMirror, para melhorar a leitura do código em C.

A validação dos exercícios será feita no próprio navegador, com regras simples para identificar respostas corretas, erros de sintaxe comuns e saídas esperadas. Esse caminho reduz custos e evita depender de servidores externos na primeira versão.

Item técnico	Definição
Plataforma	Aplicação web acessada pelo navegador.
Linguagens	HTML5, CSS3 e JavaScript.
Editor de código	Área de escrita com realce de sintaxe para linguagem C.
Armazenamento	localStorage para guardar nome, progresso e notas no navegador.
Justificativa	Evitar instalação de programas e facilitar o uso em laboratórios com restrições.

ARQUITETURA PEDAGÓGICA DO SISTEMA

10. Sequência instrucional e fluxo pedagógico

O fluxo pedagógico do Cynix segue uma sequência simples: o estudante primeiro entende o conteúdo, depois pratica, recebe feedback, corrige e só então passa para a avaliação. O foco está em aprender fazendo, mas com orientação clara a cada erro.

Etapas	Momento	O que o estudante faz
1	Entrada na unidade	Escolhe uma unidade liberada e visualiza o objetivo da aula.
2	Ativação inicial	Responde perguntas rápidas para lembrar conceitos básicos.
3	Explicação curta	Lê uma explicação simples com exemplo de código.
4	Quiz formativo	Responde questões de múltipla escolha e recebe retorno imediato.
5	Laboratório prático	Escreve um pequeno código em C para resolver um problema.
6	Teste e feedback	Executa o teste, lê o retorno do sistema e corrige o código.
7	Avaliação final	Resolve questões e um desafio prático sem dicas diretas.
8	Progresso	Recebe nota, status da unidade e orientação para avançar ou revisar.

11. Estrutura de conteúdo

O conteúdo é dividido em unidades curtas para evitar sobrecarga. Cada unidade apresenta poucos conceitos, exemplos diretos e uma prática ligada ao tema.

Unidade	Conteúdo	Prática relacionada
1	O que é algoritmo, entrada, processamento e saída.	Identificar as partes de um problema simples.
2	Estrutura básica de um programa em C e função main.	Completar um programa inicial.
3	printf, scanf e leitura de dados.	Criar um programa que recebe e mostra informações.
4	Variáveis e tipos de dados.	Declarar variáveis para idade, nota e nome.
5	Operadores aritméticos.	Calcular área, média ou valor total.

Unidade	Conteúdo	Prática relacionada
6	Estruturas condicionais.	Criar decisões com if e else.
7	Laços de repetição.	Repetir uma ação com for ou while.
8	Vetores básicos.	Guardar e percorrer uma lista pequena de valores.

12. Tipos de atividades de aprendizagem

Tipo de atividade	Como o aluno realiza	Finalidade pedagógica
Leitura guiada	Lê textos curtos com exemplos de código comentados.	Entender o conceito antes da prática.
Quiz formativo	Marca alternativas e recebe explicação para acertos e erros.	Fixar conceitos básicos.
Completar código	Preenche partes faltantes de um programa.	Perceber a estrutura do código.
Codificação prática	Escreve a solução completa no editor da plataforma.	Transformar raciocínio em código.
Correção orientada	Lê o feedback, encontra o erro e envia uma nova tentativa.	Aprender com o erro de forma segura.
Avaliação final	Resolve questões e um desafio prático com tempo definido.	Verificar se realmente aprendeu a unidade.

13. Estratégias de feedback

O feedback do Cynix não deve apenas dizer "certo" ou "errado". Ele precisa ajudar o estudante a entender o que aconteceu e o que pode fazer para melhorar.

Tipo de feedback	Exemplo de mensagem
Imediato	Seu código foi enviado. Vamos verificar a estrutura.
Indicação do erro	Parece que faltou ponto e vírgula no final de uma instrução.
Explicativo	O programa rodou, mas a conta não chegou ao resultado esperado. Confira se você multiplicou base por altura.
Dica progressiva	Dica 1: veja a fórmula. Dica 2: em C, multiplicação usa o símbolo *.
Reforço positivo	Muito bem! O código passou nos testes desta atividade.

14. Progressão do estudante

A progressão será linear, com desbloqueio por conclusão e apoio no desempenho. Isso significa que o estudante segue uma ordem planejada, mas só avança quando conclui o que é necessário.

- O estudante começa pela unidade 1 e avança para as próximas em ordem.
- O laboratório prático é liberado depois da leitura e do quiz inicial.
- A avaliação final é liberada depois da atividade prática.
- Para concluir a unidade, o estudante precisa alcançar pelo menos 80 pontos.
- Caso não alcance a nota mínima, o sistema sugere revisão e nova tentativa.

15. Estratégias de avaliação

O Cynix usa avaliação diagnóstica, formativa e somativa. Assim, o professor consegue perceber o ponto de partida do aluno, acompanhar a aprendizagem e verificar o resultado ao final da unidade.

Tipo de avaliação	Momento	Como acontece
Diagnóstica	Início da unidade ou da disciplina.	Perguntas rápidas sobre experiência prévia, conceitos e interpretação de problemas.
Formativa	Durante a trilha.	Quizzes, envio de código, feedback e novas

Tipo de avaliação	Momento	Como acontece
		tentativas sem punição direta.
Somativa	Final da unidade.	Prova com questões objetivas e desafio prático no editor.
Baseada em desempenho	Durante a prática e na avaliação final.	O estudante precisa criar um código que resolva o problema proposto.

ELICITAÇÃO E ESPECIFICAÇÃO DE REQUISITOS DO SISTEMA

16. Identificação das funcionalidades do sistema

A partir da arquitetura pedagógica, o sistema precisa cumprir funções diretamente ligadas à aprendizagem.

Pergunta da especificação	Resposta aplicada ao sistema
O que o sistema precisa fazer para apresentar o conteúdo?	Mostrar unidades com textos curtos, exemplos de código e objetivos de aprendizagem.
O que o sistema precisa fazer para permitir as atividades?	Oferecer quizzes, exercícios de completar código e editor para escrita de programas em C.
Como o sistema fornecerá feedback?	Exibindo mensagens claras no terminal virtual, com explicação do erro e dicas progressivas.
Como o sistema registrará o desempenho?	Guardando localmente notas, tentativas, unidades concluídas e status de progresso.
Como o sistema controlará a progressão?	Liberando novas etapas somente quando as anteriores forem concluídas.

Funcionalidades principais: cadastro local, exibição de conteúdo, quiz formativo, editor de código, validador de respostas, terminal de feedback, registro de progresso, avaliação final e tela de resultado.

17. Atores do sistema

Ator	Ações no sistema
Estudante	Entra na plataforma, acessa conteúdos, responde quizzes, escreve códigos, executa testes, lê feedback, refaz atividades e consulta seu progresso.
Motor de avaliação	Valida quizzes, analisa respostas de código, gera feedback, calcula nota e libera ou bloqueia etapas.

18. Requisitos funcionais

ID	Nome do requisito	Descrição do requisito
RF01	Cadastro local	O sistema deve permitir que o estudante informe nome e crie um perfil local no navegador.
RF02	Entrada no sistema	O sistema deve permitir que o estudante acesse seu perfil salvo anteriormente.
RF03	Exibição de unidades	O sistema deve apresentar as unidades de estudo em ordem de aprendizagem.
RF04	Conteúdo didático	O sistema deve mostrar explicações curtas, exemplos e objetivos da unidade.
RF05	Quiz formativo	O sistema deve disponibilizar perguntas de múltipla escolha antes da prática.
RF06	Correção do quiz	O sistema deve corrigir o quiz e mostrar explicação para cada resposta.
RF07	Editor de código	O sistema deve fornecer um editor para o estudante escrever códigos em linguagem C.
RF08	Execução de testes	O sistema deve verificar se o código atende ao problema

ID	Nome do requisito	Descrição do requisito
		proposto.
RF09	Feedback de erro	O sistema deve informar erros comuns de sintaxe e lógica em linguagem simples.
RF10	Dicas progressivas	O sistema deve apresentar dicas quando o estudante errar várias vezes a mesma atividade.
RF11	Registro de progresso	O sistema deve salvar unidades concluídas, notas e tentativas no armazenamento local.
RF12	Bloqueio de etapas	O sistema deve bloquear atividades que dependem da conclusão de etapas anteriores.
RF13	Avaliação final	O sistema deve liberar uma avaliação final com questões objetivas e desafio prático.
RF14	Cálculo de nota	O sistema deve calcular a nota final da unidade com base nas respostas e no desafio prático.
RF15	Resultado final	O sistema deve mostrar se o estudante foi aprovado na unidade ou se precisa revisar.

19. Requisitos não funcionais

ID	Categoria	Descrição
RNF01	Usabilidade	A interface deve ser simples, com botões claros, textos curtos e navegação fácil.
RNF02	Acessibilidade	A interface deve oferecer bom contraste, texto legível e feedback também por escrito, não apenas por cor.
RNF03	Desempenho	O feedback após clicar em executar deve aparecer em até 2 segundos em atividades simples.
RNF04	Portabilidade	O sistema deve funcionar nos navegadores atuais, como Chrome, Edge e Firefox.
RNF05	Confiabilidade	O sistema deve manter o progresso salvo no navegador, mesmo se a página for fechada.
RNF06	Privacidade	A primeira versão não deve enviar dados pessoais para servidores externos.
RNF07	Manutenibilidade	O código deve ser organizado em partes para facilitar correções futuras.
RNF08	Baixo custo	A solução deve evitar serviços pagos na primeira versão.

MODELAGEM DO SISTEMA

20. Diagrama de casos de uso

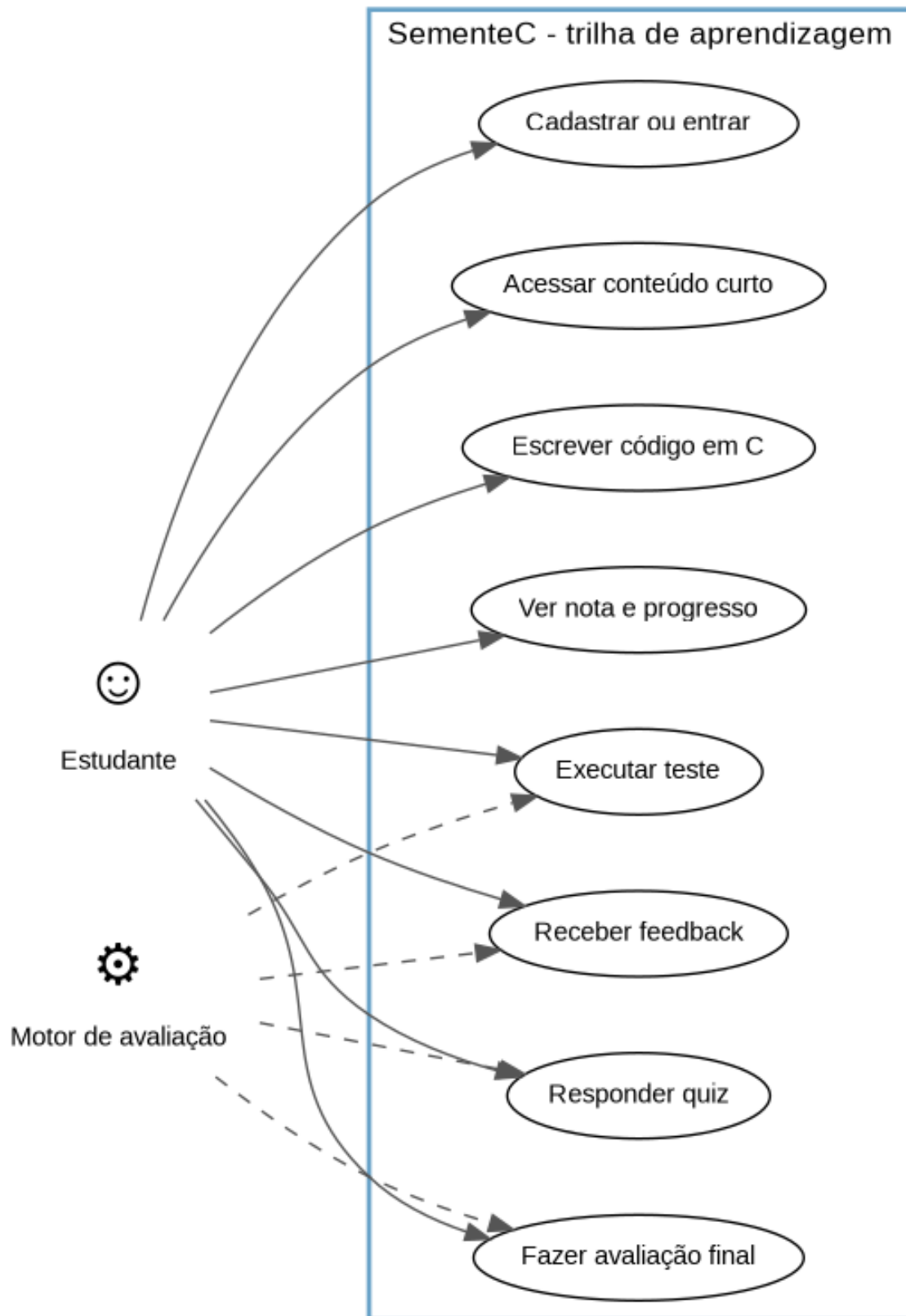


Figura 1 - Casos de uso principais do Cynix.

21. Diagrama de arquitetura geral

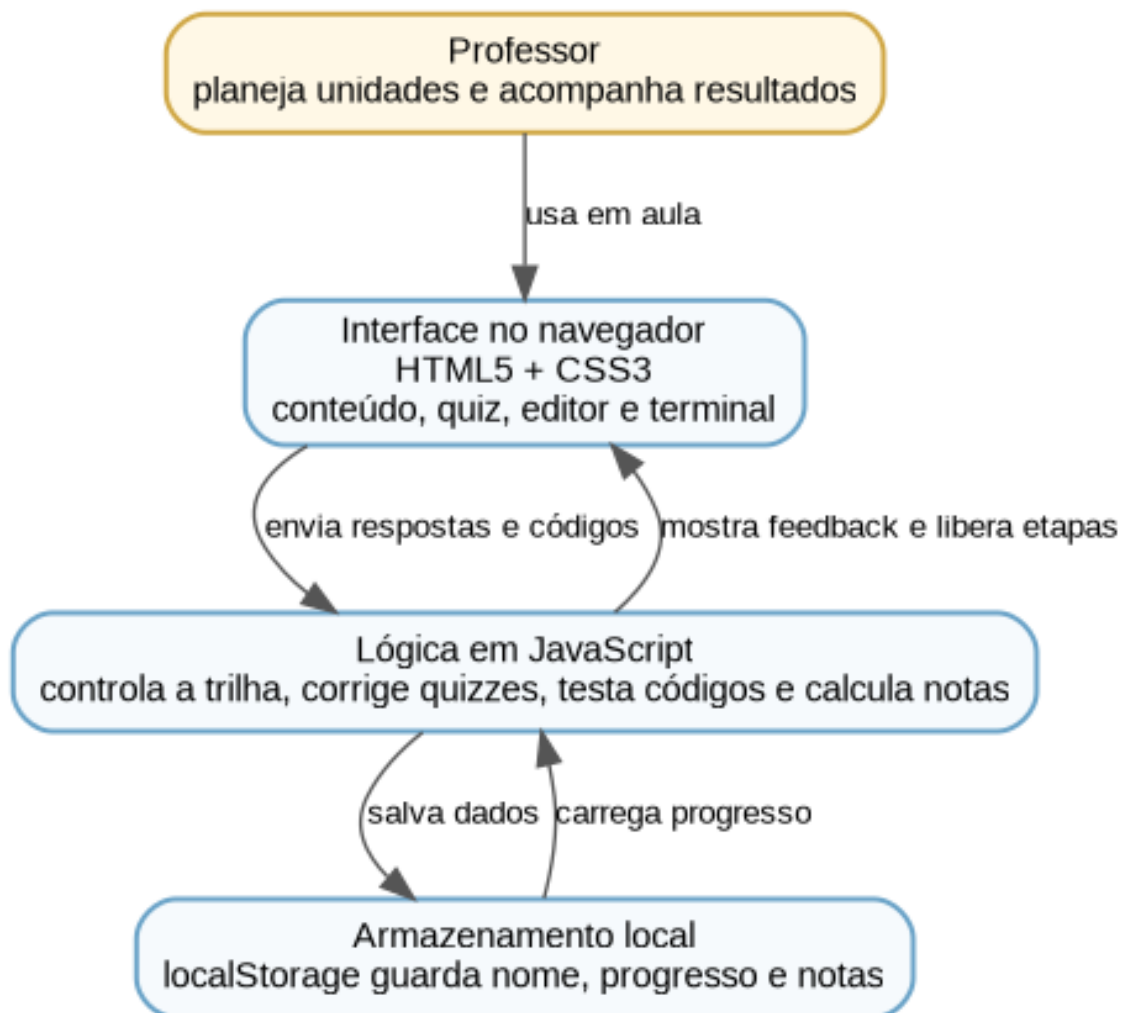


Figura 2 - Arquitetura geral da solução Cynix.

22. Diagrama entidade-relacionamento

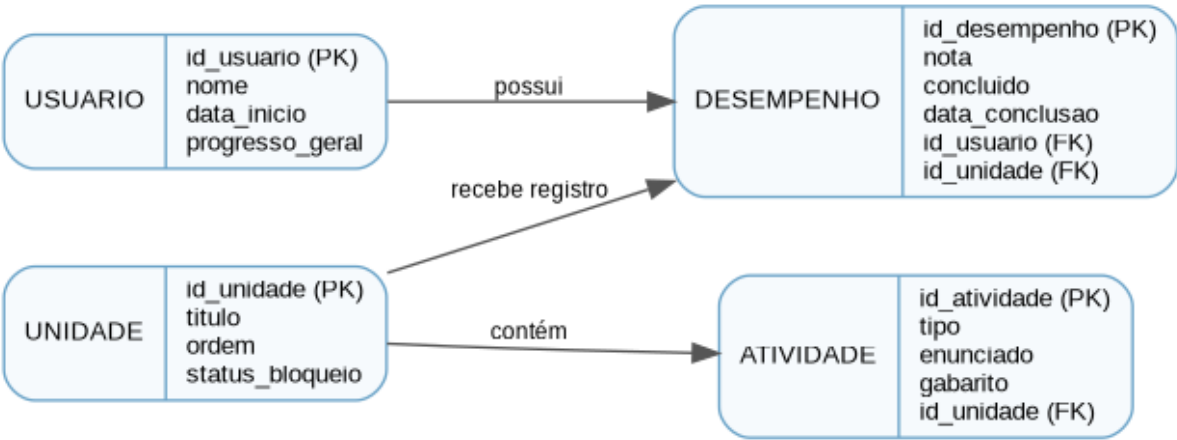


Figura 3 - Modelo entidade-relacionamento do Cynix.

23. Diagrama de atividades

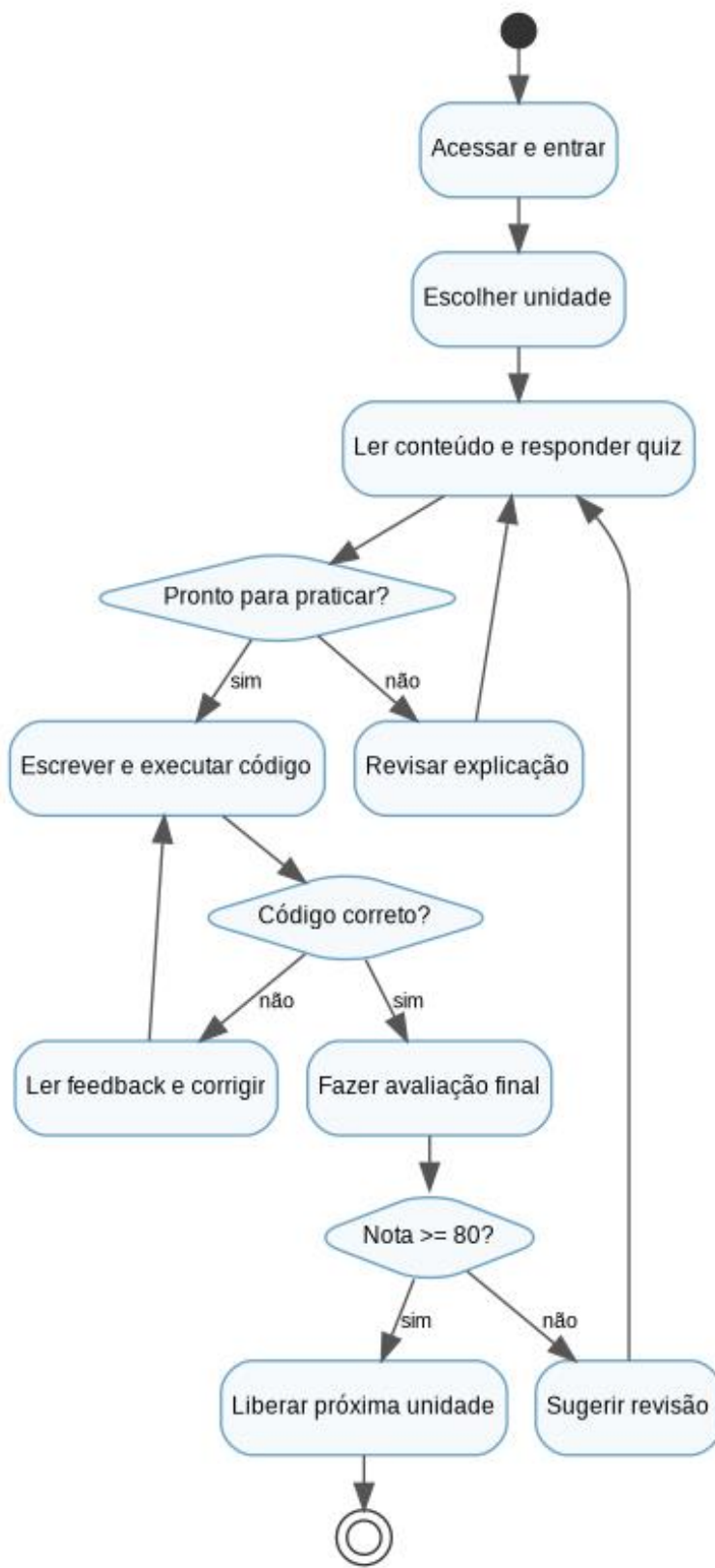


Figura 4 - Fluxo de atividade do estudante no Cynix.

24. Rastreabilidade entre objetivos, atividades e requisitos

A tabela abaixo mostra como os objetivos de aprendizagem se ligam às atividades e aos requisitos do sistema. Isso ajuda a garantir que as funcionalidades não fiquem soltas, mas ligadas ao que o estudante precisa aprender.

Objetivo	Atividade ligada	Requisitos relacionados
OA01 - Identificar entrada e saída	Leitura guiada e quiz formativo.	RF04, RF05, RF06.
OA02 - Declarar variáveis	Completar código e laboratório prático.	RF07, RF08, RF09.
OA03 - Resolver problemas com operadores	Codificação prática com teste automático.	RF07, RF08, RF10, RF11.
OA04 - Aplicar condicionais	Desafio prático e avaliação final.	RF12, RF13, RF14.
OA05 - Corrigir erros com feedback	Correção orientada e nova tentativa.	RF09, RF10, RF11, RF15.

Considerações finais

O Cynix foi planejado como uma ferramenta educacional simples, objetiva e ligada às dificuldades reais de quem está começando a programar. A proposta evita começar pela instalação de programas e coloca o foco no raciocínio, na prática e na correção dos erros.

As três etapas da atividade foram integradas em uma única documentação: primeiro foi definido o escopo do produto, depois a arquitetura pedagógica e, por fim, os requisitos e modelos do sistema. Dessa forma, o projeto fica coerente: o problema educacional justifica as funcionalidades, as funcionalidades apoiam a aprendizagem e os modelos mostram como a solução pode ser organizada.

Referências

- AMBRÓSIO, A. P. et al. Programação de computadores: compreender as dificuldades de aprendizagem dos alunos. Revista Galego-Portuguesa de Psicoloxía e Educación, v. 19, p. 185-197, 2011.
- BRASIL. Ministério da Educação. Base Nacional Comum Curricular. Brasília: MEC, 2018.
- BRASIL. Conselho Nacional de Educação. Normas sobre Computação na Educação Básica - Complemento à BNCC. Parecer CNE/CEB nº 2/2022 e Resolução CNE/CEB nº 1/2022.
- MORAIS, C. G. B.; MENDES NETO, F. M.; OSÓRIO, A. J. M. Dificuldades e desafios do processo de aprendizagem de algoritmos e programação no ensino superior: uma revisão sistemática de literatura. Research, Society and Development, v. 9, n. 10, 2020. DOI: 10.33448/rsd-v9i10.9287.